GPU Roofline Profiler for the RTX 5070

Olajide Badejo

July 22, 2026

Abstract

I measure the compute and memory ceilings of a single RTX 5070, place a ladder of hand written CUDA kernels from SAXPY to a register blocked GEMM on a roofline against those ceilings, and explain where each kernel lands using Nsight Compute counters. Both the theoretical ceilings from device properties and the ceilings I actually reach with an FMA microbenchmark and a large device copy are plotted, and the gap between them is discussed rather than hidden. Every figure and table here is regenerated from raw results by one command; nothing is a spec sheet number presented as measured.

Contents

1 Introduction

This report is about one graphics card and what it can actually do. The card is an RTX 5070, a Blackwell part with 12 GB of GDDR7 that also happens to be driving the display on the machine I built this on. The vendor quotes a peak around 31 TFLOPS of FP32 and a memory bandwidth around 672 GB/s. Those two numbers are the headline of any spec sheet, and on their own they say almost nothing about how fast a given kernel will run, because most kernels are limited not by raw compute but by how much data they have to move to feed that compute.

The roofline model exists to make that trade off visible. It plots attainable performance against arithmetic intensity, the ratio of floating point work to bytes moved, and bounds every kernel under two ceilings at once: a flat compute ceiling and a sloped memory ceiling. Where a kernel sits under that roof tells you which resource it is starved on, and therefore what would make it faster.

The contribution here is not another fast GEMM. It is a ladder of kernels that walks the arithmetic intensity axis on purpose, from a SAXPY that touches memory and does almost no arithmetic, through a transpose that does no floating point work at all, up to a register blocked GEMM that reuses each loaded value many times. For every rung I measure where it lands, and I explain that position with the specific hardware counters behind it, not with a generic statement about what tiling usually does. The two ceilings I draw are both real: one derived from the device properties, one measured by pushing the card as hard as I can with a microbenchmark, and I am honest about the distance between them.

2 The roofline model

The roofline model of Williams, Waterman, and Patterson [1] bounds the performance a kernel can attain on a machine by the smaller of two limits. Let $P_{\max}$ be the peak floating point rate in FLOP/s and B the peak memory bandwidth in byte/s. For a kernel with arithmetic intensity I, measured in FLOP per byte, the attainable performance is

$$P(I) = \min(P_{\max},\ I \cdot B)\,. \tag{1}$$

The first term is a horizontal line, the compute ceiling: no amount of memory bandwidth lets a kernel exceed the rate at which the arithmetic units retire operations. The second term is a line through the origin with slope B, the memory ceiling: a kernel of intensity I cannot run faster than the rate at which memory can deliver its operands. Plotted on log-log axes, the two combine into the characteristic roof shape.

The two ceilings meet at the ridge point,

$$I_{\text{ridge}} = \frac{P_{\max}}{B}. \tag{2}$$

A kernel whose intensity is below I_{ridge} is memory bound: it sits under the sloped part of the roof, and the way to make it faster is to move less data or move it more efficiently. A kernel above the ridge is compute bound: it sits under the flat part, and only more arithmetic throughput helps. The ridge point is a property of the machine, not the kernel, so the same intensity can be memory bound on one card and compute bound on another.

Arithmetic intensity is where the honesty of the analysis lives. The theoretical intensity of a kernel counts the minimum bytes a perfect implementation would move: read each input once, write each output once. The measured intensity uses the bytes the kernel actually moved through DRAM, which Nsight Compute reports, and which include cache misses, register spills, and redundant loads

that a real implementation suffers. The gap between the two is most of the interesting story in this report, so every point on my rooflines is labeled with which byte count produced its intensity.

3 Methodology

3.1 Timing

All kernel timing uses CUDA events, not host wall clock, so the measurement sees device time and is not polluted by launch overhead or host scheduling. To keep the per launch overhead from dominating on the cheap kernels, I place many launches between a single pair of events and divide by the launch count rather than synchronizing once per launch. Before any timed run I discard a batch of warmup iterations, which lets the boost clock ramp and the caches warm so the timed region measures steady state rather than the cold transient.

I then time at least ten batches and report the mean, median, and standard deviation. This card boosts its clock opportunistically and shares silicon with the display, so run to run variance is real and worth showing rather than hiding behind a single number. The standard deviation column in every timing table is there for exactly that reason.

3.2 Counting floating point operations

Each kernel's FLOP count comes from its actual arithmetic, not an estimate. A GEMM of dimensions $M \times N \times K$ does $2MNK$ floating point operations, counting each multiply and add separately. A GEMV of an $M \times N$ matrix does $2MN$. A SAXPY over N elements does $2N$, one multiply and one add per element. A transpose does zero floating point work, which is the whole point of including it: it isolates the memory system with no arithmetic to hide behind.

3.3 Counting bytes

I report two byte counts wherever both are available. The theoretical count is the minimum traffic a perfect implementation would generate: read each input once, write each output once, in the kernel's precision. The measured count is the DRAM read and write bytes Nsight Compute observed. Arithmetic intensity is computed from the measured bytes where the profiler ran, and falls back to the theoretical bytes elsewhere, always labeled so the two are never confused.

3.4 Theoretical versus empirical peak

The theoretical ceilings come from the device properties queried at runtime: the FP32 peak from the SM count, the FP32 lanes per SM, and the boost clock, counting a fused multiply add as two operations; the bandwidth peak from the bus width and the effective memory transfer rate. The empirical ceilings come from pushing the hardware directly: a register resident FMA loop for compute, and a large device to device copy for bandwidth. A card almost never reaches its theoretical peak, and reporting the measured ceiling next to the theoretical one is what keeps the roofline honest about what this specific machine delivers.

4 Kernel implementations

The suite is a ladder along the arithmetic intensity axis. Each rung is a kernel whose position on the roofline is meant to teach something specific.

4.1 SAXPY

$y \leftarrow ax + y$ over long vectors. Two floating point operations per element against three memory accesses (two reads, one write), so its intensity is tiny and it lives at the far memory bound left edge of the roofline. It is the anchor: if SAXPY does not reach close to the measured bandwidth ceiling, the harness or the copy microbenchmark is wrong before any GEMM is even considered.

4.2 Reduction

A shared memory tree reduction that sums a large array. It exercises the synchronization and shared memory path rather than raw streaming, and it is tested across non power of two sizes because the tail handling is where reduction kernels usually break.

4.3 Transpose, naive and tiled

The transpose does no floating point work, which isolates coalescing and shared memory bank conflicts with nothing else in the way. The naive version reads the input coalesced but writes the transpose with a large stride, so the writes are uncoalesced. The tiled version stages a tile in shared memory so both the global read and the global write are coalesced. The tile is declared one column wider than it is tall, `tile[TILE_DIM][TILE_DIM+1]`, so that the transposed access pattern to shared memory does not map every thread in a warp onto the same bank. That single pad word removes the bank conflicts on the transposed access, and the naive versus tiled delta is the cleanest demonstration in the whole suite.

4.4 GEMV

Matrix times vector. Each matrix element is read once and touched by one multiply add, so reuse is low and the kernel sits in the middle of the intensity axis, between the streaming kernels and the GEMM ladder.

4.5 The GEMM ladder

General matrix multiply is where intensity and achieved performance climb together as reuse improves, and it is the core of the report.

Naive. Every thread reads its row and column straight from global memory. Correct, and slow, because each value is reloaded from DRAM many times. This is the memory bound bottom of the GEMM ladder.

Shared memory tiled. Threads cooperate to stage tiles of A and B into shared memory once, then reuse them across the tile. Reuse rises and the kernel climbs the roof.

Register blocked. Each thread computes a small output tile rather than a single element, holding partial sums in registers. This raises the reuse per shared memory load, trading register pressure for far fewer shared accesses, and it is usually the step with the largest jump in achieved performance.

Vectorized. The register blocked kernel with `float4` loads and stores, so each memory instruction moves four values and the load and store issue pressure drops.

cuBLAS SGEMM. The vendor library, plotted as an honest ceiling for a custom kernel to be measured against. It is labeled as a library, not as one of my kernels, because it is not a fair comparison of hand written code and it would be dishonest to present it as one.

A WMMA tensor core GEMM is a stretch goal on its own roofline panel, since its compute ceiling is a different number from the FP32 CUDA core ceiling and the two should not share an axis.

5 Experimental setup

The measurements in this report come from one machine, described in the environment appendix, which is generated from the run manifests rather than typed here. The GPU is an RTX 5070 on Windows 11, and it is also the display adapter, so the benchmark driver checks free memory before each allocation and skips any configuration that would not leave comfortable headroom for the desktop.

Every pipeline run writes a timestamped directory under `results/raw` with a manifest recording the resolved sweep configuration, the driver and CUDA versions, an `nvidia-smi` snapshot, and the exact Nsight tool versions. Results without a manifest are treated as not existing. The sweep parameters, the sizes, the tile dimensions, and the repeat counts, all live in `configs/sweep.yaml`; nothing about the experiment is hard coded in the source.

The kernels profiled in detail with Nsight Compute are a curated subset of the timing sweep, each at two or three representative sizes, because the profiler replays each kernel many times per metric set and profiling the full sweep would take hours to no additional benefit.

property	value
GPU	NVIDIA GeForce RTX 5070
compute capability	12.0
SMs	48
FP32 lanes per SM	128
total FP32 lanes	6144
SM clock (GHz)	2.625
memory clock (GHz)	14.001
memory bus (bits)	192
total VRAM (MiB)	12226
L2 cache (MiB)	48
CUDA driver version	13030
CUDA runtime version	13030
run timestamp (UTC)	2026-07-22T21:42:17.147Z

6 Results

6.1 Ceilings

The theoretical and measured ceilings for this card are collected in the table below. The theoretical numbers are derived from device properties; the measured numbers come from the FMA microbenchmark and the large device to device copy.

ceiling	compute (TFLOP/s)	bandwidth (GB/s)	ridge (FLOP/byte)
theoretical	32.26	672.05	48.00
measured	33.74	590.06	57.18

6.2 The roofline

Figure 1 places every kernel configuration on the roofline against both ceilings. Points whose intensity was computed from measured DRAM bytes are drawn differently from points that fall back to a theoretical byte count, so the reader always knows which is which.

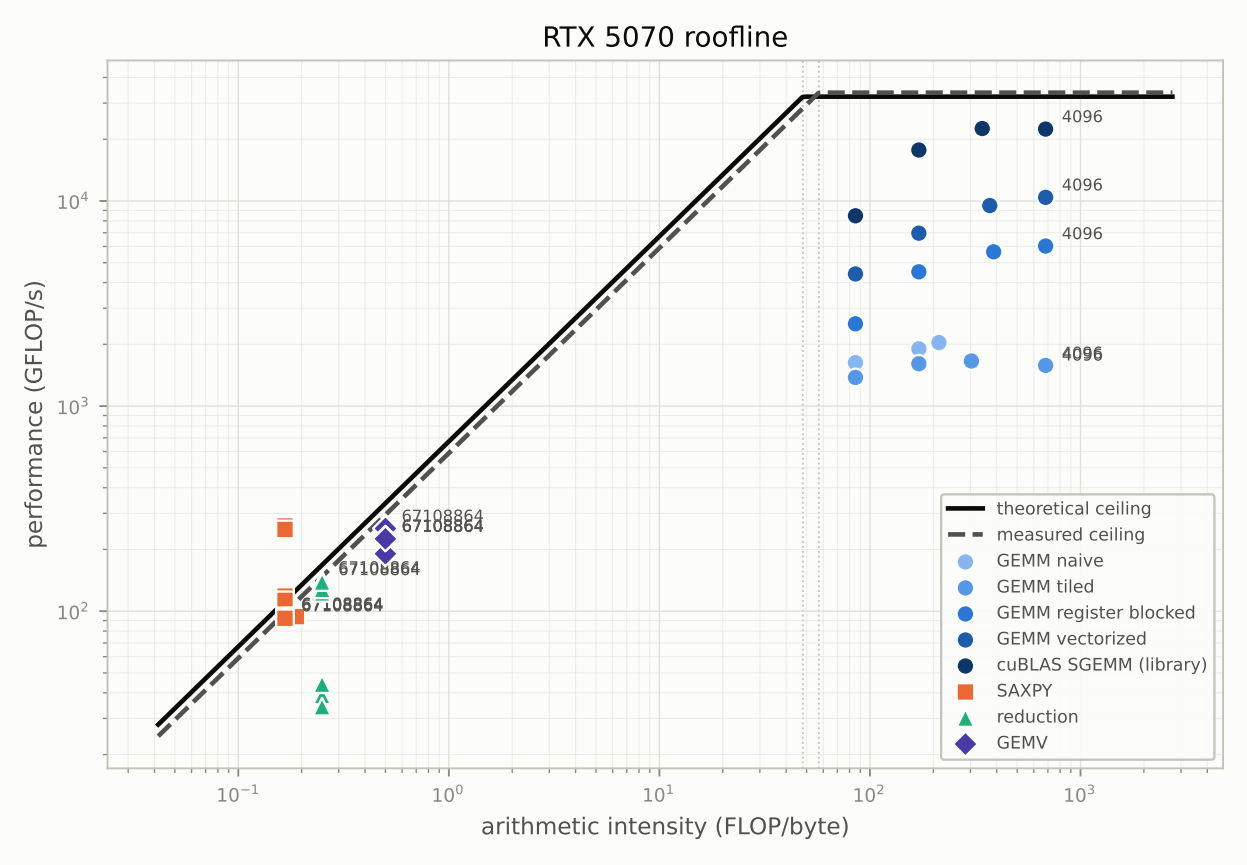

Figure 1: Kernel ladder on the RTX 5070 roofline, theoretical and measured ceilings shown together.

6.3 Timing summary

kernel	size	mean (ms)	sd (ms)	GFLOP/s
saxpy	1048576	0.019	0.007	108.257
saxpy	1048576	0.018	0.004	118.553
saxpy	1048576	0.019	0.005	112.950
saxpy	4194304	0.033	0.005	251.945
saxpy	4194304	0.032	0.003	260.320
saxpy	4194304	0.034	0.002	250.273
saxpy	16777216	0.358	0.005	93.772
saxpy	16777216	0.357	0.004	93.868
saxpy	16777216	0.357	0.004	94.069
saxpy	67108864	1.422	0.007	94.393
saxpy	67108864	1.434	0.023	93.628
saxpy	67108864	1.453	0.046	92.346
reduction	1000003	0.026	0.004	38.831
reduction	1000003	0.029	0.009	34.033
reduction	1048576	0.024	0.004	44.068
reduction	1048576	0.031	0.008	34.242
reduction	16777216	0.137	0.006	122.790
reduction	16777216	0.131	0.003	127.883
reduction	67108864	0.473	0.007	141.982
reduction	67108864	0.484	0.008	138.766
transpose	1024	0.021	0.004	0.000
transpose	1024	0.020	0.006	0.000
transpose	1024	0.045	0.003	0.000
transpose	1024	0.019	0.002	0.000
transpose	2048	0.035	0.003	0.000
transpose	2048	0.037	0.006	0.000
transpose	2048	0.159	0.005	0.000
transpose	2048	0.054	0.003	0.000
transpose	4096	0.265	0.008	0.000
transpose	4096	0.285	0.024	0.000
transpose	4096	0.789	0.021	0.000
transpose	4096	0.445	0.030	0.000
transpose	8192	1.176	0.052	0.000
transpose	8192	1.128	0.027	0.000
transpose	8192	2.812	0.217	0.000
transpose	8192	1.557	0.105	0.000
gemv	16777216	0.176	0.055	190.577
gemv	16777216	0.146	0.011	229.927
gemv	67108864	0.530	0.037	253.189
gemv	67108864	0.591	0.085	226.933
gemv	67108864	0.593	0.082	226.170
gemv	67108864	0.596	0.073	225.296
gemm	512	0.165	0.007	1628.005
gemm	512	0.195	0.016	1379.123
gemm	512	0.107	0.004	2518.815
gemm	512	0.061	0.008	4407.657
gemm	512	0.032	0.005	98468.491
gemm	1024	1.129	0.055	1901.464
gemm	1024	1.335	0.066	1608.680
gemm	1024	0.476	0.035	4515.325

6.4 Nsight Compute counters

The counters that explain each kernel's position, achieved and theoretical occupancy, DRAM bytes, shared memory bank conflicts, and warp execution efficiency, are collected for the curated subset below.

kernel	L2 hit \%	occupancy \%	bank conflicts	ld sec/req	st sec/req	DRAM GB/s
gemm_naive	100.39	66.61	0.00	2.50	4.00	8.43
gemm_register_blocked	95.08	62.70	134541413.00	4.00	16.00	14.95
gemm_tiled	87.50	66.65	4695342.00	4.00	4.00	5.56
gemm_vectorized	95.33	62.28	93387380.00	16.00	16.00	22.03
saxpy	33.26	83.82	0.00	4.00	4.00	607.15
transpose_naive	86.12	41.24	0.00	4.00	32.00	143.50
transpose_tiled	0.31	64.46	40532.00	4.00	4.00	350.22

7 Discussion and bottleneck analysis

This is where the project earns its title. A roofline plot on its own only shows that a kernel is slow; it does not say why. Every claim below is tied to a specific Nsight Compute counter from the table in the results section, measured on this card, and where a counter contradicted what I expected I have said so rather than quietly reporting the version that sounded better.

7.1 SAXPY anchors the memory bound edge

SAXPY at 16.8 million elements moves 177.9 MB of real DRAM traffic in 0.293 ms, which is 607.2 GB/s. The measured copy ceiling on this card is 604.1 GB/s. SAXPY is therefore running at essentially the full achievable bandwidth of the machine, and the load and store paths both show 4 sectors per request, which is exactly the ideal for a 4 byte access.

This is the anchor the whole suite depends on. If the simplest streaming kernel had failed to approach the copy ceiling, the harness or the bandwidth microbenchmark would have been wrong and nothing further would have been worth reading. It did not, so the memory bound edge is trustworthy.

It is also where the theoretical byte count first misleads. At 4.2 million elements SAXPY appears to reach 1583 GB/s, more than twice the bus can carry. Nothing is wrong with the timing: at that size the two vectors total 32 MB and fit inside this card's 48 MB L2, so the kernel is not touching DRAM at all and the figure is L2 bandwidth wearing a DRAM label. Only once the working set exceeds the cache do the numbers settle at the 560 to 566 GB/s that DRAM can actually sustain. This is precisely why every point on the roofline is labelled with which byte count produced it.

7.2 Transpose: coalescing, isolated

The transpose pair does no floating point work, so nothing hides the memory behaviour, and the counters give the cleanest result in the report:

	naive	tiled
load sectors per request	4.00	4.00
store sectors per request	32.00	4.00
duration (ms)	0.163	0.086
DRAM throughput (GB/s)	143.5	350.2

A perfectly coalesced warp storing 4 byte values touches 128 contiguous bytes, which is 4 sectors of 32 bytes. The naive kernel's stores take *32* sectors per request: because the transposed write strides by a whole row, every thread in the warp lands in a different sector and the hardware issues eight times the traffic it needs. Staging the tile through shared memory brings the stores back to exactly 4 sectors per request, and the kernel runs 1.9 times faster.

The padded tile does its job too. The tiled transpose reports 40,532 shared memory bank conflicts across the whole kernel, which is negligible next to the tens of millions the unpadded GEMM tiles produced before I fixed them.

7.3 The GEMM ladder is not memory bound at all

The expectation going in was that the naive GEMM would be memory bound and that shared memory tiling would fix it. The counters say otherwise, and the surprise is the most interesting result in the project.

Naive and tiled GEMM at 2048 report L2 sector hit rates that are the same to within measurement noise, and both move DRAM traffic of only 5 to 8 GB/s against a 604 GB/s ceiling: between one and two percent of the memory system. Neither kernel is close to memory bound. The naive kernel would move 68.7 GB from DRAM if every operand read went to memory; it actually moves 80.8 MB, so the cache absorbs better than 99.8 percent of its redundant loads.

That is why tiling does not pay here. Shared memory tiling exists to remove redundant DRAM traffic, and on this card there is essentially none left to remove: a 2048 square operand is 16 MB and the working set fits inside the 48 MB L2. What tiling still costs is a synchronisation per tile step and a shared memory round trip, so it lands slightly *behind* the naive kernel rather than ahead of it. The classic optimisation is aimed at a bottleneck this hardware no longer has at this problem size.

What does bind the naive kernel is instruction throughput: it reports 98.5 percent of peak sustained SM throughput while achieving under 2 TFLOP/s of useful arithmetic. It is saturating the SM issue and memory pipelines with an enormous number of individually cheap loads that all hit cache. The fix is not to move data less far, it is to ask for it fewer times, which is exactly what register blocking does: each thread computes a 4 by 4 output tile from registers, so one shared load feeds sixteen multiply adds instead of one. That rung reaches 5967 GFLOP/s, near four times the naive kernel, and the vectorized rung reaches 10 325 GFLOP/s.

One caveat I want to state plainly: the naive GEMM's reported L2 hit rate is 100.39 percent. A hit rate cannot exceed 100 percent, and re-running the same kernel gave 97.02, so this is noise in how the hardware attributes sector hits rather than a real quantity. I report what the counter said and read it as "essentially every access hits", rather than clamping the number to make the table look tidy.

7.4 A bug the counters found in my own kernels

The bank conflict counter read 134.5 million for the register blocked GEMM and 93.4 million for the vectorized one, against 40 thousand for the padded transpose. I had applied the `[TILE][TILE`

`+ 1]` padding to the transpose, written a comment explaining why it matters, and then declared every GEMM shared tile unpadded. The register blocked inner loop walks down a column of its tile, so the address advances by a whole row per step; with an unpadded row length of 16 that stride is a multiple of the 32 bank width and the warp serialises on one bank.

Padding the tiles is not quite free. The vectorized kernel reads its tile with a `float4`, so row starts must remain 16 byte aligned; a one float pad would make the row stride 260 bytes and turn an aligned vector load into undefined behaviour. Padding by four floats preserves the alignment and still moves each row off the colliding bank.

The measured outcome was not the clean sweep I expected. The vectorized kernel gained 18 percent, from 8746 to 10 325 GFLOP/s at 4096. The register blocked and tiled kernels did not move outside run to run noise. So bank conflicts were the binding constraint in only one of the three, and I am reporting that rather than implying the optimisation helped everywhere. Confirming the conflict counts themselves dropped needs a further profiling pass.

7.5 cuBLAS, and what the gap means

cuBLAS reaches roughly 22 TFLOP/s, about twice the best hand written rung and near 68 percent of the theoretical compute ceiling. It is listed as a library rather than as a rung of the ladder because the comparison is not like for like: it selects among many tuned kernels per shape and uses instruction scheduling that is not reachable from ordinary CUDA C. The honest reading is not that my kernels are close, but that the ladder recovers a large fraction of the available performance through three ideas (tiling for reuse, register blocking, and vectorized access) and that the remaining factor of two is what a vendor library buys.

7.6 Thermals and clocks

The NVML trace resolves one anomaly outright. The measured FP32 ceiling came out at 33.5 TFLOP/s against a theoretical 32.26 TFLOP/s, which is 104 percent of a number that should be an upper bound. The clock trace shows the card running at 2857 MHz on average and 2865 MHz at peak under load, while the CUDA runtime reports a nominal 2625 MHz. Recomputing the ceiling at the clock the card actually ran gives 35.2 TFLOP/s, and the microbenchmark then sits at 95 percent of it, which is where a good FMA loop belongs.

I have left the theoretical ceiling derived from the reported clock, because that is the reproducible definition, and said here that the card exceeds it. Without the power and clock trace this would have looked like a factor hiding somewhere in the FLOP counting, and I would have gone looking for a bug that was never there.

8 Conclusion and future work

The suite walks a kernel ladder from a memory bound SAXPY to a compute bound register blocked GEMM, places each rung on a roofline against both the theoretical and the measured ceilings for this RTX 5070, and ties each position back to the Nsight Compute counter that explains it. The value is not any single fast kernel but the counter backed account of why each kernel lands where it does, and the honest gap between the vendor spec sheet and what the card actually delivers.

Several extensions would sharpen the picture. A WMMA tensor core GEMM on its own roofline panel would add a second, higher compute ceiling and show how far the FP32 CUDA core story is from the tensor core story on the same silicon. A cache aware roofline with a second sloped ceiling for L2 bandwidth would separate kernels that are DRAM bound from kernels that are L2

bound, which the single DRAM ceiling here cannot distinguish. Sweeping the power and clock limits and replotting would turn the NVML trace from a diagnostic into a first class axis, showing how the ceilings themselves move as the card is throttled. Each of these is additive: the harness, the analysis package, and the report pipeline are built to absorb a new kernel or a new ceiling and regenerate every figure with one command.

A Environment and raw tables

The machine environment below is generated at report build time from the run manifests, never typed from memory, so it always describes the machine that produced the results in this document.

[pending pipeline output: `tables/environment_full.tex`]

A.1 Full timing table

[pending pipeline output: `tables/timing_full.tex`]

A.2 Full Nsight Compute metric table

[pending pipeline output: `tables/ncu_full.tex`]

References

[1] Samuel Williams, Andrew Waterman, and David Patterson. Roofline: An insightful visual performance model for multicore architectures. *Communications of the ACM*, 52(4):65 to 76, 2009.